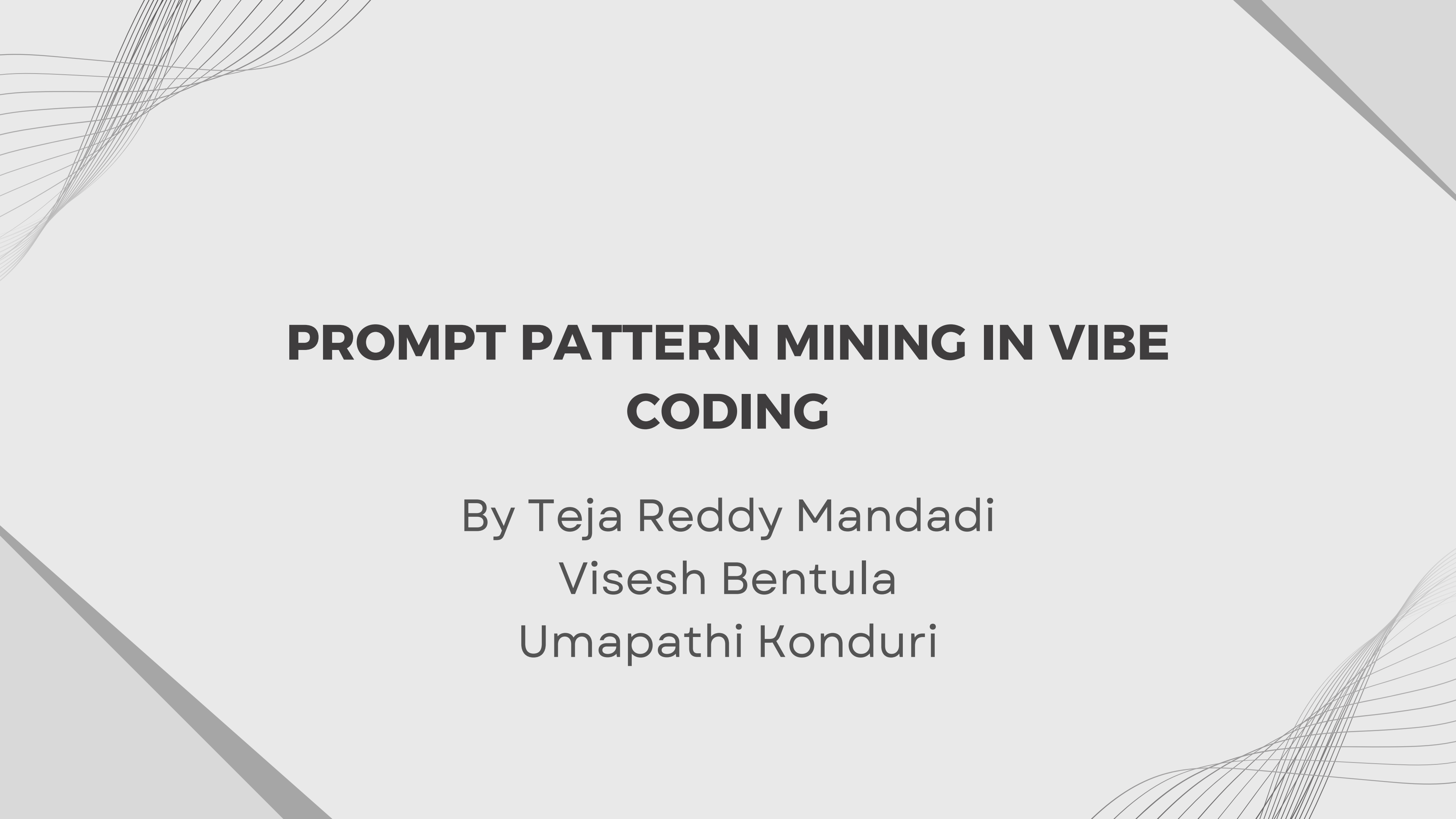




# **PROMPT PATTERN MINING IN VIBE CODING**

By Teja Reddy Mandadi  
Visesh Bentula  
Umapathi Konduri

# INTRODUCTION

- 
- Large Language Models (LLMs) are increasingly integrated into software development workflows.
  - Developers rely on conversational prompting to generate, refine, and debug code.
  - However, prompt quality and structure vary significantly across users.
  - Our project investigates which measurable prompt patterns are associated with more successful coding interactions.
  - We frame this as an empirical software engineering study using real-world developer-LLM conversations.
- 

# BACKGROUND - LLMS IN SOFTWARE ENGINEERING

01

LLMs such as ChatGPT and Codex are used for:

- Code generation
- Bug fixing
- Refactoring
- Documentation writing

02

Studies show productivity improvements but also highlight risks such as hallucinated or insecure code.

03

Research from empirical software engineering and ACM/IEEE venues shows that output quality is highly sensitive to prompt design.

04

Security-focused studies emphasize the importance of structured, well-defined prompts to reduce vulnerabilities.

Key Insight:

Understanding how developers structure prompts is critical to improving reliability and effectiveness of AI-assisted coding.

# BACKGROUND - VIBE CODING & PROMPTING BEHAVIOR

01

“Vibe coding” refers to interactive, iterative collaboration between developer and LLM.

02

Developers refine prompts by:

- Adding constraints
- Providing examples
- Requesting formatting
- correcting model outputs

03

Prior studies show prompting strategy significantly impacts code correctness and usability.

04

However, most existing work uses controlled experiments rather than large-scale real-world conversation analysis.

Research Gap:

There is limited quantitative evidence on which prompt patterns consistently lead to successful coding outcomes in natural developer workflows.

# RELATED WORK I – DEVELOPER-LLM INTERACTION



Springer – Empirical Software Engineering (2024)  
→ Analyzes ChatGPT use in GitHub PRs and Issues  
→ Identifies iterative refinement patterns



DevGPT (Xiao et al., arXiv)  
→ Introduces large-scale dataset of developer-ChatGPT conversations



Wu et al. (ACM Conference)  
→ Extracts prompt patterns from DevGPT dialogues



Prompting Techniques for Secure Code Generation (arXiv)  
→ Evaluates prompting effects on secure coding outcomes

GAP: EXISTING WORK IDENTIFIES PATTERNS BUT LACKS MULTIVARIABLE STATISTICAL MODELING OF STRUCTURAL PROMPT FEATURES



# RELATED WORK II – DEVELOPER-PROMPT ENGINEERING TECHNIQUES



IEEE ICSME 2025 – Prompting Matters  
→ Controlled evaluation of prompting techniques



Elsevier (ScienceDirect) – Unleashing the Potential  
of Prompt Engineering  
→ Structured prompts improve reliability



Improving ChatGPT Prompt for Code Generation  
(arXiv)  
→ Comparison of prompting strategies



Prompt Alchemy (arXiv)  
→ Automatic iterative refinement improves  
performance

MOST STUDIES RELY ON CONTROLLED EXPERIMENTS, NOT REAL-WORLD WORKFLOW MINING.

# RELATED WORK III – BROADER CONTEXT & CONCEPTUAL FOUNDATIONS



SN Computer Science (Springer) – LLMs for Code Generation Review  
→ Systematic overview of LLM use in software engineering



ACM – LLM Security Hardening Study  
→ Highlights reliability and vulnerability risks



Frontiers in Education – Prompt Engineering as a Skill  
→ Establishes prompt design as an emerging competency



Prompt Pattern Catalog (Technical Report)  
→ Formalizes reusable prompt structures

THESE WORKS ESTABLISH IMPORTANCE OF PROMPTING, BUT NONE PERFORM LARGE-SCALE STATISTICAL MINING OF STRUCTURAL PROMPT FEATURES IN REAL-WORLD CODING WORKFLOWS.

# RESEARCH GAP

01

## From prior literature:

- Prompt engineering improves output
  - Developers iteratively refine prompts
  - LLM output quality varies significantly
- 

02

## Missing:

- Feature-level mining of prompt structure
- Statistical modeling of prompt success
- Regression analysis controlling for context



# RESEARCH OBJECTIVES

✓ Extract measurable prompt features from real conversations.

---

✓ Quantitatively analyze structural and behavioral prompt characteristics.

---

✓ Model statistical predictors of successful coding outcomes.

---

✓ Provide interpretable, reproducible results.

---

# RESEARCH QUESTIONS

## RQ1:

Which structural prompt features (length, constraints, examples, formatting instructions) are associated with successful coding conversations?

## RQ2:

How does iterative behavior (number of turns, refinement prompts, correction cycles) influence success probability?

## RQ3:

After controlling for metadata (language, model type, source context), which features remain statistically significant?

# RESEARCH METHODOLOGY OVERVIEW

---

Empirical software engineering approach:

1. Data preprocessing
2. Feature extraction
3. Success labeling
4. Statistical analysis

Pipeline designed to be reproducible and interpretable.

# DATA COLLECTION PLAN

---

## DATASET:

DevGPT dataset (Xiao et al.)

## PROCESSING STEPS:

- Parse JSON conversations
- Remove duplicates and incomplete records
- Normalize and clean textual content
- Structure data into:
  - Turn-level dataset
  - Conversation-level dataset

## UNIT OF ANALYSIS:

- Individual prompts
- Entire conversation threads

# FEATURE EXTRACTION DESIGN



We extract measurable prompt features:

- Prompt length
- Presence of constraints
- Use of examples
- Output formatting instructions
- Number of iterations
- Correction/refinement prompts

All features are text-based and reproducible.



# SUCCESS DEFINITION

---

Because execution validation is outside scope, we define success using:

- Explicit positive confirmation signals
- Reduced iteration depth
- Presence of final code output
- Lack of corrective follow-ups

We use conservative labeling to avoid overestimating success.



# EXPERIMENTAL PLAN

## Step 1: Descriptive statistics

- Distribution of prompt features
- Iteration depth patterns

## Step 2: Correlation analysis

- Identify preliminary associations

## Step 3: Logistic regression modeling

- Predict success probability
- Control for metadata variables

## Step 4: Robustness checks

- Alternative success definitions
  - Subgroup analysis by language/context
-

# EXPECTED OUTCOMES

---

- Empirical evidence linking prompt structure to success.
- Feature-level statistical modeling of real conversations.
- Practical insights for developer prompt engineering.

Contribution to:

- Prompt engineering research
- Empirical software engineering
- Human-AI collaboration studies



# REFERENCES

---

## Peer-Reviewed (7)

1. An Empirical Study on Developers' Shared Conversations with ChatGPT in GitHub PRs and Issues.

Empirical Software Engineering (Springer), 2024.

<https://link.springer.com/article/10.1007/s10664-024-10540-x>

2. Wu et al. (2024). ChatGPT Chats Decoded: Uncovering Prompt Patterns for Superior Solutions in Software Development.

ACM Conference Proceedings.

<https://doi.org/10.1145/3643991.3645069>

3. Usage of Large Language Model for Code Generation Tasks: A Review.

SN Computer Science (Springer), 2025.

<https://doi.org/10.1007/s42979-025-04241-5>

4. Prompting Matters: Assessing the Effect of Prompting Techniques on LLM-Generated Class Code.

IEEE International Conference on Software Maintenance and Evolution (ICSME 2025).

<https://doi.org/10.1109/ICSME64153.2025.00083>

5. Unleashing the Potential of Prompt Engineering for Large Language Models.

Elsevier / ScienceDirect, 2025.

<https://www.sciencedirect.com/science/article/pii/S2666389925001084>

# REFERENCES

---

6. Large Language Models for Code: Security Hardening and Adversarial Testing.  
ACM Conference Proceedings.

<https://dl.acm.org/doi/10.1145/3576915.3623175>

7. Prompting Techniques for Secure Code Generation: A Systematic Investigation.  
arXiv (2024).

<https://arxiv.org/abs/2407.07064>

## **Supporting References:**

8. DevGPT: Studying Developer-ChatGPT Conversations.  
Xiao et al., arXiv (2023).

<https://arxiv.org/abs/2309.03914>

9. Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation.  
Ye et al., arXiv (2025).

<https://arxiv.org/abs/2503.11085>

10. Improving ChatGPT Prompt for Code Generation.  
Liu et al., arXiv (2023).

<https://doi.org/10.48550/arXiv.2305.08360>

11. A Prompt Pattern Catalog to Enhance Prompt Engineering.  
Schmidt et al., Technical Report (PDF).

<https://www.dre.vanderbilt.edu/~schmidt/PDF/PLoP-patterns.pdf>

12. Prompt Engineering as a Next-Gen Skill.  
Frontiers in Education, 2024.

<https://www.frontiersin.org/articles/10.3389/feduc.2024.1366434/full>

The image features abstract line art in the top-left and bottom-left corners. These elements consist of numerous thin, dark grey lines that curve and sweep across the page, creating a sense of movement and depth. The lines are more densely packed in some areas, forming soft, cloud-like shapes, while in others they are more sparse and delicate.

**THANK YOU**

---